Journey with Python

Introduction to Programming

Sheet 1 — Programming Basics

Devansh Mishra
BCA (Cloud Computing & Cyber Security), IIMT University
GitHub: auditordevansh LinkedIn: auditordevansh

Welcome to the first sheet of the *Journey with Python* series!

Before we dive into the world of programming, I would like you to pause and reflect on a few fundamental questions. These may seem simple or even irrelevant at first glance, but thinking about them carefully will make your learning journey far smoother and more effective.

1. Warm-Up Questions

Take your time with the following questions. You are encouraged to use any resource available — search engines, books, or AI assistants — to explore your answers.

> **Q1.** What is a computer, and how does it actually process complex tasks and problems?
>
> **Q2.** What are the fundamental building blocks of a computer or its computation power?
>
> **Q3.** Why do we need specific programming languages (such as Python, Java, C/C++, Rust, etc.) when we can seemingly interact with computers through graphical interfaces or plain English commands?

2. Problem-Solving Intuition

Set programming and computer science aside for a moment, and focus on a more fundamental skill: **problem solving**.

> **Reflection Question:** How do you approach a problem? What is your intuition in the very first moment you encounter one? Take your time to answer and honestly self-evaluate.

2.1. A Classic Problem: Greatest Common Divisor (GCD)

Consider the following problem statement:

Problem Statement: Given two natural numbers a and b (where $a, b \in \mathbb{N}$), find their Greatest Common Divisor $\gcd(a, b)$.

Before writing any code or choosing any approach, we must first *understand* and *decompose* the problem. This strategy is formally known as **problem decomposition**.

2.2. Approach: Problem Decomposition

Fortunately, this problem statement is straightforward. We are given two natural numbers and are asked to find their GCD. To do so efficiently, we need a clear, ordered sequence of steps — in other words, an **algorithm**.

Definition. An *algorithm* is a finite sequence of well-defined instructions used to solve a problem or perform a computation.

2.3. Algorithm: Step-by-Step Solution

Let us trace through the algorithm for the specific values $a = 10$ and $b = 15$.

Step 1. Input the two numbers.
Let $a = 10$ and $b = 15$.

Step 2. Recall the definition of GCD.
The *Greatest Common Divisor* of two numbers is the largest positive integer that divides both numbers without leaving a remainder.

Step 3. Factorize both numbers.

$$\text{Factors of } a = 10 : \{1,\ 2,\ 5,\ 10\}$$

$$\text{Factors of } b = 15 : \{1,\ 3,\ 5,\ 15\}$$

Step 4. Identify the common factors.

$$\text{Common factors of 10 and 15} : \{1,\ 5\}$$

Step 5. Select the greatest among the common factors.

$$\text{Greatest common factor} = 5$$

Step 6. Conclude.

$$\gcd(10,\ 15) = 5$$

We now have a working algorithm that holds for *any* valid values of a and b.

Exercise: Stop here and try to apply this algorithm manually for at least two different pairs of values, e.g., $(a, b) = (12, 18)$ and $(a, b) = (7, 28)$. Write out each step carefully.

3. From Algorithm to Code

> **Key Question:** Can we teach a computer to follow these same steps automatically?

Yes — absolutely! The process of translating an algorithm into a programming language is called **coding** (or **implementation**). This is precisely where Python comes in.

In the sections below, we will write Python programs that implement the GCD algorithm in three progressively refined ways — mirroring exactly how a programmer thinks and improves their solution.

3.1. Approach 1: Naïve / Brute-Force Method

The most direct translation of our algorithm: iterate over all integers from 1 to $\min(a, b)$, keep track of every common divisor, and return the largest one found.

```
#


#  GCD  |  Approach 1 : Brute Force
#  Idea : Check every number from 1 to min(a, b).
#         Store all common divisors, then pick the largest.
#



def gcd_brute(a, b):
    """Return GCD of two natural numbers using brute force."""
    common_divisors = []                    # Step 3 & 4 from the
       algorithm

    for i in range(1, min(a, b) + 1):    # check 1, 2, ..., min(a,
       b)
        if a % i == 0 and b % i == 0:    # i divides both a and b?
            common_divisors.append(i)    # collect it

    return max(common_divisors)           # Step 5 & 6 -- largest
       one


#         Test


a, b = 10, 15
print(f"GCD({a}, {b}) = {gcd_brute(a, b)}")    # Expected: 5
print(f"GCD(12, 18) = {gcd_brute(12, 18)}")    # Expected: 6
print(f"GCD(7, 28)  = {gcd_brute(7, 28)}")     # Expected: 7
```

Listing 1: GCD – Naïve brute-force approach

> **Observation.** This works correctly, but it is *inefficient*. For very large numbers (say, $a = 10^9$, $b = 10^9 - 1$), the loop runs nearly a billion iterations. We need a smarter approach.

3.2. Approach 2: Euclidean Algorithm

The **Euclidean Algorithm** is one of the oldest and most elegant algorithms in mathematics. It is based on the key observation:

$$\gcd(a,\ b)\ =\ \gcd(b,\ a \bmod b)$$

We keep replacing (a, b) with $(b,\ a \bmod b)$ until the remainder is 0. At that point, the non-zero value is the GCD.

```
#


#  GCD  |  Approach 2 : Euclidean Algorithm (Iterative)
#  Key insight : gcd(a, b) == gcd(b, a % b)
#  Stop when   : the remainder becomes 0
#



def gcd_euclidean(a, b):
    """Return GCD using the Euclidean algorithm."""
    while b != 0:           # keep going until remainder is 0
        a, b = b, a % b    # replace (a, b) with (b, a mod b)
    return a                # when b == 0, a holds the GCD


#         Trace for gcd(10, 15)


# Step 1 : a=10, b=15        new a=15, new b=10%15=10    (wait,
#          Python evaluates right side first, so:)
#          a=10, b=15        a=15,  b=10 % 15 = 10
# Step 2 : a=15, b=10        a=10,  b=15 % 10 = 5
# Step 3 : a=10, b=5         a=5,   b=10 % 5  = 0
# b == 0, so return a = 5

#         Test


print(f"GCD(10, 15) = {gcd_euclidean(10, 15)}")    # 5
print(f"GCD(12, 18) = {gcd_euclidean(12, 18)}")    # 6
print(f"GCD(7, 28)  = {gcd_euclidean(7, 28)}")     # 7
```

Listing 2: GCD – Euclidean algorithm (iterative)

3.3. Approach 3: Recursive Euclidean Algorithm

The same logic can be expressed even more concisely using **recursion** — a function that calls itself with a simpler version of the problem.

```
#


#  GCD  |  Approach 3 : Euclidean Algorithm (Recursive)
#  Base case       : b == 0       return a
#  Recursive case  : return gcd(b, a % b)
#



def gcd_recursive(a, b):
    """Return GCD using recursion."""
    if b == 0:                          # base case: nothing left
        to divide
        return a
    return gcd_recursive(b, a % b)     # recursive step


#         Test


print(f"GCD(10, 15) = {gcd_recursive(10, 15)}")    # 5
print(f"GCD(12, 18) = {gcd_recursive(12, 18)}")    # 6
print(f"GCD(7, 28)  = {gcd_recursive(7, 28)}")     # 7


#         Bonus: Python's built-in


import math
print(f"\nVerification via math.gcd:")
print(f"math.gcd(10, 15) = {math.gcd(10, 15)}")    # 5
```

Listing 3: GCD – Euclidean algorithm (recursive)

3.4. Summary: Comparing the Three Approaches

Approach	Time Complexity	Lines of Code	Best Used When
Brute Force	$O(\min(a, b))$	~6	Learning / small numbers
Euclidean (iter.)	$O(\log(\min(a, b)))$	~4	Production code
Euclidean (recur.)	$O(\log(\min(a, b)))$	~3	Elegant / concise code

Key Takeaway. All three programs produce the *same correct answer*. The difference lies in **efficiency** and **readability**. As you grow as a programmer, you will learn to choose the right tool for the right context — and that judgment is what separates a good programmer from a great one.

References & Inspirations

The concepts, pedagogical structure, and problem-solving philosophy presented in this sheet draw inspiration from the following courses and educators. Learners are strongly encouraged to explore these resources alongside this series.

[1] NPTEL — Programming, Data Structures and Algorithms Using Python

Instructor: Prof. Madhavan Mukund
Affiliation: Department of Computer Science, Chennai Mathematical Institute (CMI)
Platform: NPTEL / SWAYAM, Government of India

The core algorithmic intuition in this sheet — particularly the emphasis on problem decomposition, step-by-step thinking before coding, and the GCD as a first algorithm — is directly inspired by Prof. Mukund's opening lectures of this course. His calm, rigorous approach to building up from first principles is the model this series aspires to follow.

Topics covered in the course: Conditionals, loops, functions, lists, strings, tuples, searching & sorting algorithms, dynamic programming, backtracking, exception handling, file I/O, dictionaries, linked lists, binary search trees.

Access: `onlinecourses.nptel.ac.in` · `YouTube Playlist`

[2] CS50x — Introduction to Computer Science

Instructor: Prof. David J. Malan (Gordon McKay Professor, Harvard University)
Platform: Harvard University / edX **(Free & self-paced)**

The framing of computational thinking, abstraction, and algorithm design used in this sheet is shaped by the philosophy of CS50 — Harvard's largest course. CS50's approach of teaching "how to think algorithmically and solve problems efficiently" resonates deeply with the goals of this series, particularly the Warm-Up Questions in Section 1.

Topics covered: Computational thinking, abstraction, algorithms, data structures, encapsulation, resource management, security, software engineering, web development. Languages: C, Python, SQL, JavaScript, HTML, CSS.

Access: `cs50.harvard.edu/x` · `edX: CS50 Program`

[3] Kaggle Learn — Python & Intro to Programming

Platform: Kaggle (a subsidiary of Google) **(Free)**

Kaggle's micro-courses serve as a practical, hands-on complement to the theoretical foundations built in this series. The Python micro-course covers syntax, functions, loops, lists, dictionaries, and practical programming — all topics this series will progressively address. Kaggle's interactive notebook environment is especially recommended for running and experimenting with the code examples presented in Sheet 1.

Recommended courses:

- Intro to Programming *(no prior experience needed)*
- Python *(syntax, functions, data types)*
- Pandas *(data manipulation — for later sheets)*

Access: kaggle.com/learn/python · kaggle.com/learn/intro-to-programming

A Note on Learning. These references are not prerequisites — they are companions. This series is designed to stand on its own, but exploring these courses in parallel will significantly deepen your understanding. Prof. Mukund's NPTEL course, in particular, follows a nearly identical problem-first philosophy and is *highly recommended* for any serious learner.

End of Sheet 1 — Journey with Python Series